VaLoRe

ML-powered real estate valuation platform

Milestone 2

INFO9023 - Machine Learning System Design

Antoine DECKERS (s2170999)
Hoang Linh BUI (s2503303)
Duy Vu DINH (s2401627)

April 1st, 2026

Milestone 2 Deliverable

- Project value creation and ML canvas
- Dependency management migration from PIP to UV
- EDA extension - Feature importance
- Local ML modeling pipeline
- Deployment of ML pipeline
- Model serving and API

1. Project Overview

Topic: ML-powered real estate valuation platform.

Goal: build an end-to-end machine learning system that aims to value a residential property given some training data containing house features

Value Creation:

- For buyers: quickly estimate fair property price → avoid overpaying
- For sellers: set competitive listing prices → faster transactions
- For real estate agents → support data-driven pricing decisions
- For banks / investors → assist in property risk and valuation analysis

1. Project Overview | ML Canvas

-

Machine Learning Canvas

Product:	Authors:	Date:	Version:
VaLoRe	Antoine, Hoang Linh, Duy Vu	March 29, 2026	V1

Background

The main user need is simple: estimating a property's expected price quickly and consistently. In real life, price estimation can take time, may depend too much on manual judgment, and can vary between people.

Value proposition

- For buyers: quickly estimate fair property price
- For sellers: set competitive listing prices
- For real estate agents: support data-driven pricing decisions
- For banks / investors: assist in property risk and valuation analysis

Objectives

- Explore and process the given data
- Build a regression model
- Serve the model through API
- Deploy an end-to-end system on Cloud that allow CI/CD, automated model training, model serving, online prediction, performance monitoring, automated model retraining.

Solution

A MLOps application with two main sides, including serving and training

Feasibility

- Data is tabular with moderate data size
- ML techniques performs well on tabular data and is practical to serve
- Cloud stack is manageable because it stays within one ecosystem, Google Cloud

- Potential risk: quality of the data

Data

Housing Price in King County, USA between May 2014 and May 2015
- Source: Kaggle
- Datapoints: 21,614
- Data type: tabular / structured
- Number of features: 21, including `price`
- Target variable: price
- No missing values and duplicates

Based on the aim of the project, the dataset will be stored in BigQuery as the main cloud data source, and the raw CSV exist in GCS

Metrics

RMSE, MAE, R^2

Evaluation

Offline: LS / TS splits
- Model selection: K-fold CV
- Model evaluation: Hold-out method

Online: Control deployment and feedback

Modeling

Experiment - Baseline model selection
- Data splitting: LS (80%), TS (20%)
- Model zoo: Dummy Regressor, Linear Regression, KNN, Random Forest, XGBoost,etc.
- Hyper-param tuning: K-fold CV
- Model comparison: Hold-out method
  + Train on LS
  + Performance evaluate on TS
  + Baseline model is chosen based on the best performance regarding metric

Model training
- Data splitting: LS (80%), TS (20%)
- Regression model: baseline model
- Model selection: K-fold CV
- Model evaluation: Hold-out method
  + Train on LS
  + Performance evaluate on TS

Inference

Thanks to light-weight ML model, API, container as well as Cloud service, the system could infer the new instance in real-time

Feedback

- Model feedback
- Serving feedback
- Pipeline feedback
- Milestone feedback

Project

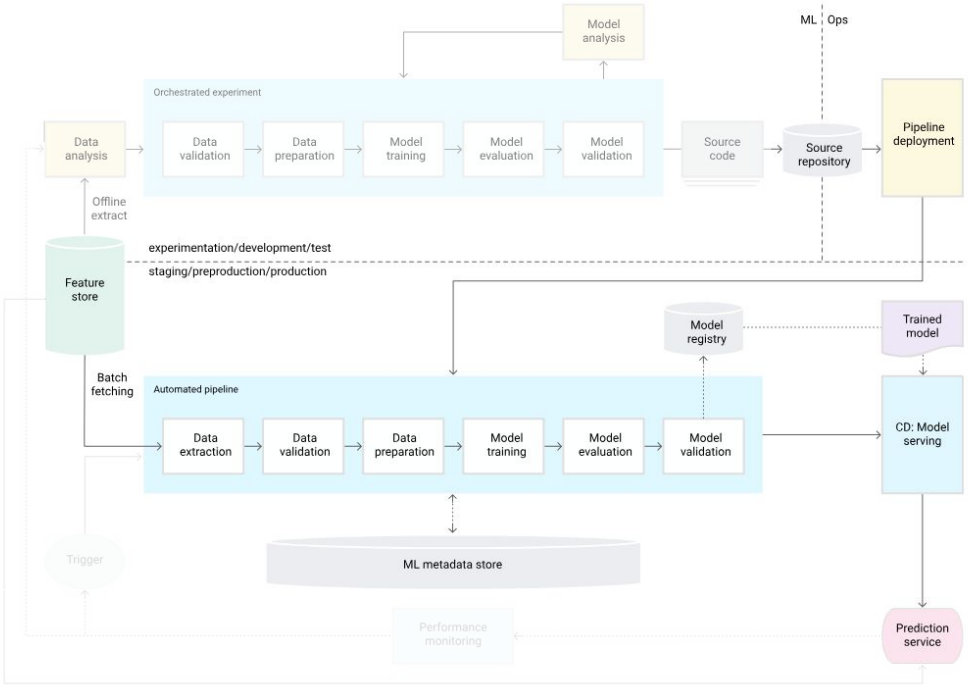
Team member:
- Antoine DECKERS
- Hoang Linh BUI
- Duy Vu DINH

- Deliverables: GitHub repo, Dashboard, Report slides, also milestone's objectives

Roadmap:
- Milestone 1:
  + Git, CI/CD, GitFlow setup
  + EDA
  + Preliminary modeling
- Milestone 2:
  + modeling improvement
  + Model serving and deploy for API
  + ML pipeline
- Milestone 3:
  + Dashboard
  + Integration
  + Automation

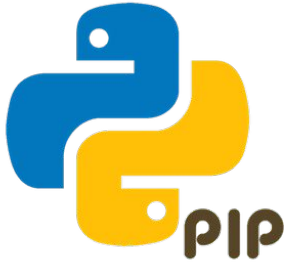
Machine learning canvas from Made With ML by Goku Mohandas
License: CC BY-SA 4.0

1. Project Overview | MLops Pipeline



A. **Feature store**:
   a. Local data source: .csv
   b. Cloud data source: BigQuery, GCS
B. **Data analysis** (EDA)
   a. Data understanding
   b. Data description
   c. Uni-variate analysis
   d. Multi-variate analysis
   e. Importance features
C. **Model analysis**: baseline model selection (update)
D. **Orchestrated experiment** (update)
   a. Data validation
   b. Data preparation
   c. Model training
   d. Model evaluation
   e. Model validation
E. **Source repository**: GitHub repo
F. **Pipeline deployment**: Vertex AI Pipeline
G. **Automated pipeline**: Kubeflow
   a. Data extraction
   b. Data validation
   c. Data preparation
   d. Model training
   e. Model evaluation
   f. Model validation
H. **ML metadata store**: GCS Bucket
I. **Model registry**
J. **CD: Model serving**
K. **Prediction service**
L. **Performance monitoring**
M. **Trigger**

2. Dependency Management | from PIP to UV

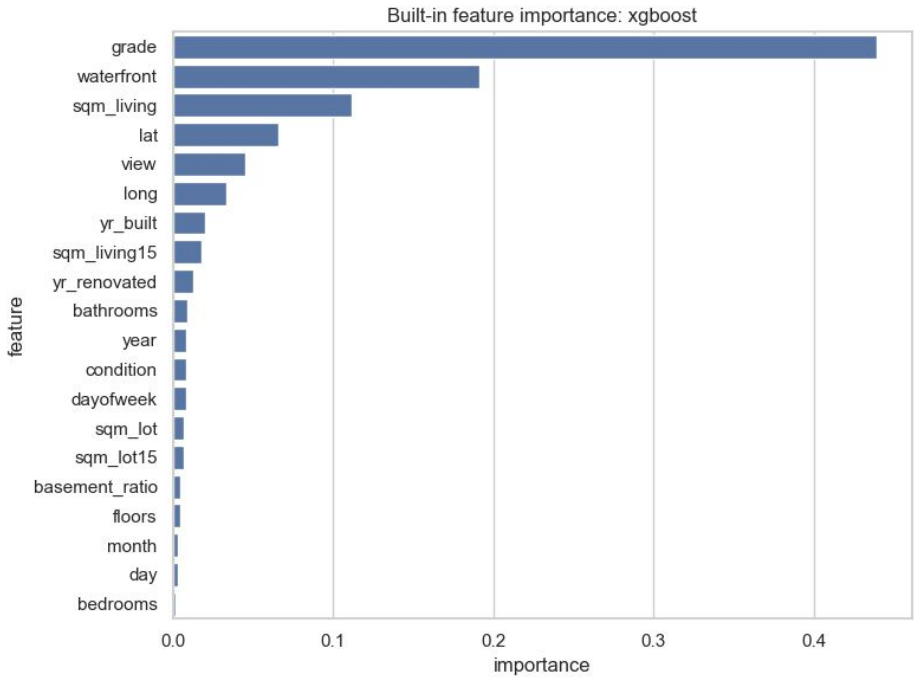


requirements.txt

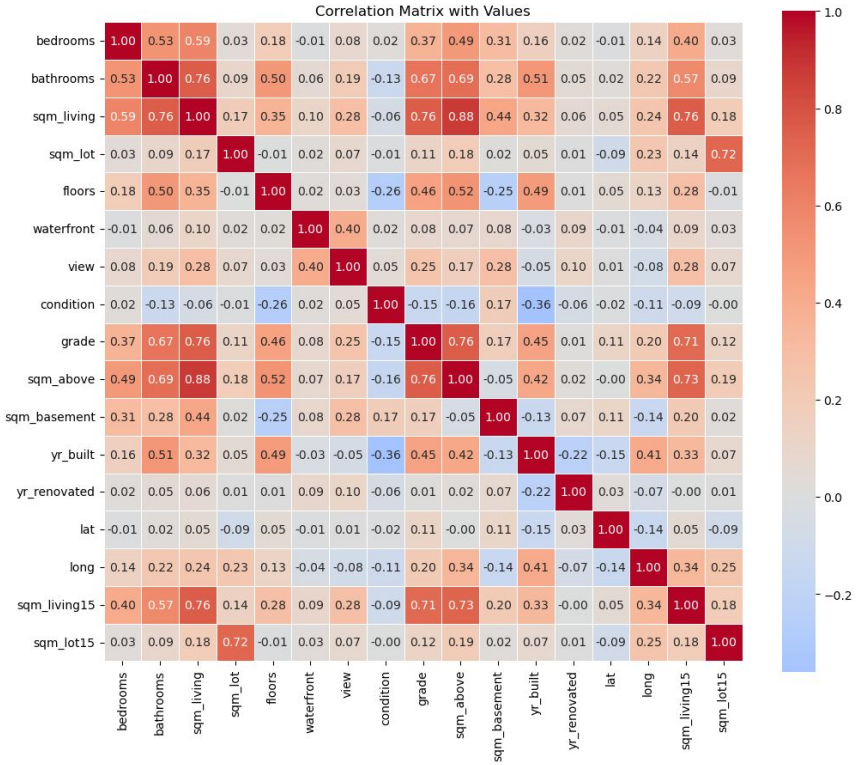
pyproject.toml
uv.lock

3. EDA | Feature Importance

- **Grade** is the most important feature (~45%) → construction quality strongly impacts price
- **Waterfront** significantly increases property value → premium location factor
- **Location** features (latitude, view) play a key role in pricing
- **Size-related** features (sqm_living) contribute moderately
- ***Temporal and minor*** features (year, dayofweek, condition) have low impact

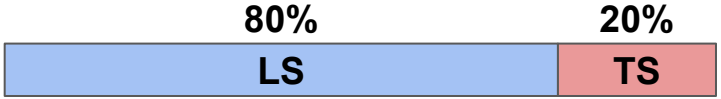


3. EDA | Multi-variate Analysis



4. Model Analysis | Baseline Model Selection

Data splitting



Pipeline

Data Loading → **Data Preprocessing** → **Hyper-param Tuning** → **Model Comparison** → **Baseline Model Selection**

Load data from Biquery table

Data cleaning
Feature Engineering

Method: 5-fold CV on LS
Metric: RMSE

Method: Hold-out test
- Train on LS
- Evaluate on TS

Metric: RMSE
Add. Metrics: MAE, R2

Chosen model is the model with **the lowest RSME on TS**

File: `**notebooks/experimentation.ipynb**`

4. Model Analysis | Baseline Model Selection

Model Zoo

Linear Regression

Param grid: None

XGBoost

Param grid
- "n_estimators": [100, 300],
- "learning_rate": [0.05, 0.1, 0.2],
- "max_depth": [4, 6, 8],

KNN

Param grid
- "n_neighbors": [5, 9, 15],
- "weights": ["uniform", "distance"]

Random Forest

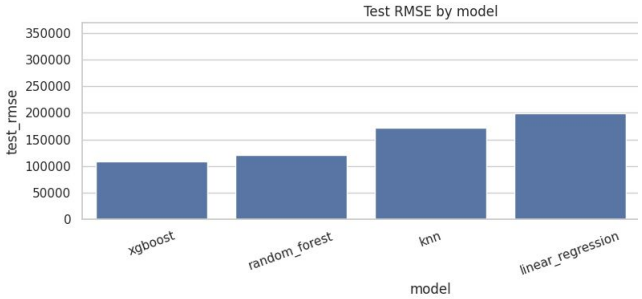
Param grid
- "n_estimators": [100, 300],
- "max_depth": [None, 10, 20],
- "min_samples_leaf": [1, 2],

File: `**notebooks/experimentation.ipynb**`

4. Model Analysis | Baseline Model Selection

Model	LS - Cross-val. - RMSE	TS - MAE	TS - RMSE	TS - R2
XGBoost	**126273.76**	**61871.37**	**108789.75**	**0.904127**
Random Forest	131329.16	68212.74	120751.42	0.881884
KNN	182848.66	96650.79	172364.45	0.759332
Linear Regression	NaN	124767.64	198791.33	0.679877

⇒ **XGBoost** is chosen as **Baseline model**



File: `**notebooks/experimentation.ipynb**`

5. Orchestrated Experiment | Overall Pipeline

Overview

Orchestrated Experiment

Data Loading → **Data Validation** → **Data Pre-processing** → **Model Training** → **Model Evaluation** → **Model Validation**

Data loaded from BigQuery table

`src/modeling/data_loading.py`

Assume all datapoints are valid

Data cleaning
Feature Engineering

`src/modeling/preprocessing.py`

Data splitting
Model selection
Model training

`src/modeling/splitting.py`
`src/modeling/model_selection.py`
`src/modeling/training.py`

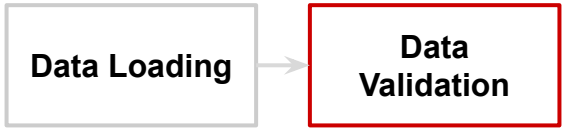
Performance estimation

`src/modeling/evaluation.py`

File: `**src/modeling/run_training.py**`

5. Orchestrated Experiment | Data Validation

Assumption: All datapoints are **VALID**



Data loaded from BigQuery table

5. Orchestrated Experiment | Data Preprocessing

Variable removal

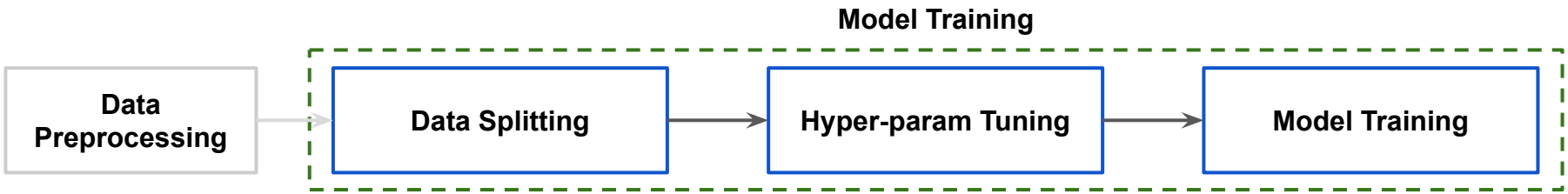
- Drop id: zero predictive power
- Drop zipcode: lat / long is more accurate

Compositional variables (add up to a constant)

- Sqm_living = sqm_above + sqm_basement
- Problem : redundant signal (noise) and can impact model stability / interpretability
- Choice : keep sqm_living and introduce basement_ratio (sqm_basement / sqm_living), drop sqm_basement/above.

File: `**src/modeling/preprocessing.py**`

5. Orchestrated Experiment | Model Training



Data splitted into:
- Learning samples (LS): 80%
- Test samples (TS): 20%

Method: 5-fold CV on LS
Metric: RMSE

`sklearn.model_selection.GridSearchCV`

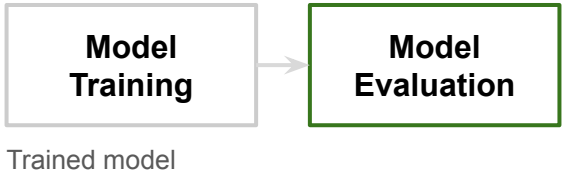
Method: Hold-out test
- Train on LS

XGBoost

Param grid
- "n_estimators": [100, 300],
- "learning_rate": [0.05, 0.1, 0.2],
- "max_depth": [4, 6, 8],

File:
`src/modeling/splitting.py`
`src/modeling/model_selection.py`
`src/modeling/training.py`

5. Orchestrated Experiment | Model Evaluation



The model is **evaluated on the TS** in order to **estimate the model performance**

Metrics: **RMSE**, MAE, R2

File: `**src/modeling/evaluation.py**`

5. Orchestrated Experiment | Model Validation

Several questions:

- Does the metric exceed the **thresholds**?
- Is it better than the **baseline**?
- Is it better than the **model currently running** in production?
- Is it too **slow**?
- Is it too **heavy**?
- Is it **stable**?
- Does it comply with **business rules**?

6. Automated Pipeline | Pipeline

Orchestrated experiment	→	**Automated pipeline**
	→	`src/vertex/run_pipeline.py`
	→	`src/vertex/data_ingestion.py`
`src/modeling/preprocessing.py`	→	`src/vertex/preprocessing_component.py`
`src/modeling/splitting.py`	→	`src/vertex/splitting_component.py`
`src/modeling/model_selection.py + /training.py`	→	`src/vertex/training_component.py`
`src/modeling/evaluation.py`	→	`src/vertex/evaluation_component.py`

File: `**src/vertex/run_pipeline.py**`

7. Pipeline Deployment | Vertex Pipeline

Orchestrated experiment

`src/modeling/preprocessing.py`
`src/modeling/splitting.py`
`src/modeling/model_selection.py + /training.py`
`src/modeling/evaluation.py`

Automated pipeline
→ `src/vertex/run_pipeline.py`
→ `src/vertex/data_ingestion.py`
→ `src/vertex/preprocessing_component.py`
→ `src/vertex/splitting_component.py`
→ `src/vertex/training_component.py`
→ `src/vertex/evaluation_component.py`

Local Machine

Docker image
(`Dockerfile.vertex`)

- uv installation

- `pyproject.toml`
- `uv.lock`

- `src/`
  - `config.py`
  - `vertex/`
  - `modeling/`

(Additional component…)

→

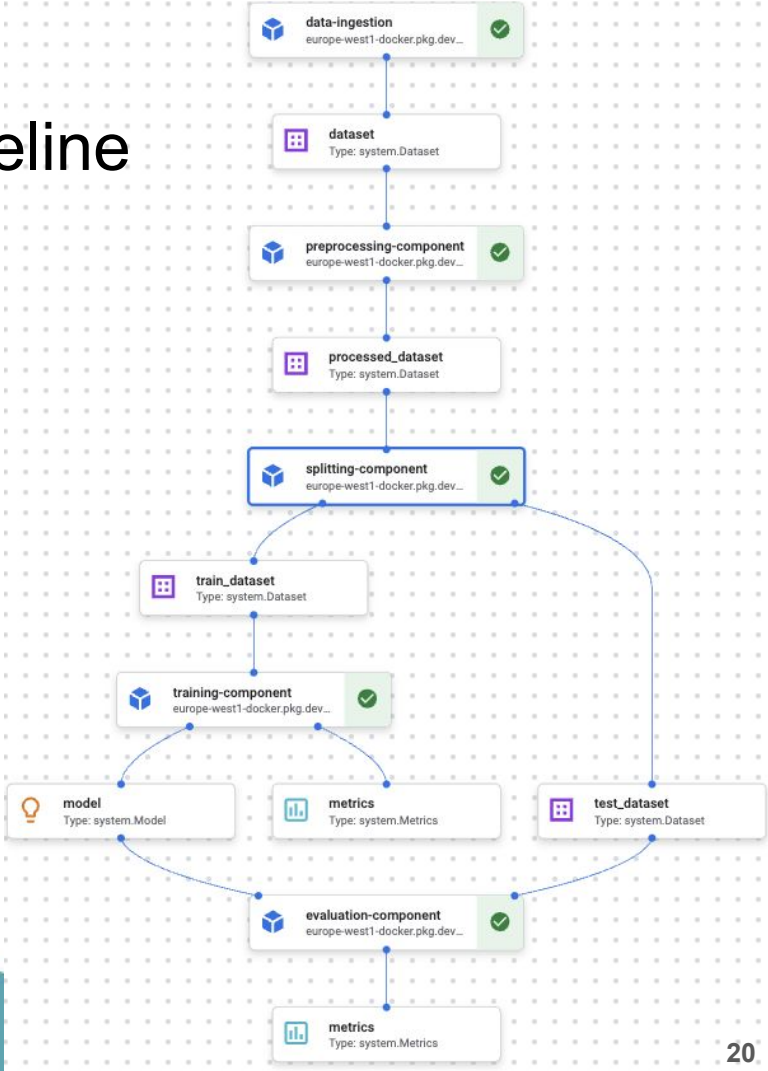
Artifact Registry repo

File: `**Dockerfile.vertex**`

7. Pipeline Deployment | Vertex Pipeline

1. **data-ingestion**: export the BigQuery table to GCS as parquet files, then register that *saved as a parquet dataset artifact path*
2. **preprocessing-component**: read the raw data (parquet files), apply our project preprocessing, *write it as a parquet dataset artifact path*
3. **splitting-component**: load the processed data, split it into train and test dataframes, *save them as two separate parquet dataset artifact paths*
4. **training-component**: load the training split, run model selection, fit the final model, *save the trained model artifact*, *log training-related metrics*
5. **evaluation-component**: load the model artifact saved by the training component, load the test split, *compute final evaluation metrics*, *log those metrics to Vertex*

File: `**src/vertex/run_pipeline.py**`



8. ML Metadata Store

GCS Bucket

Objects | Configuration | Permissions | Protection | Lifecycle | Observability | Inventory Reports | Operations

Buckets > mlsd-valore-2026-0001 > pipeline_root_valore > 826612837630 > valore-pipeline-20260331180311

Create folder | Upload | Transfer data | Create batch operations New | Other services | Learn

Sort and filter | Filter Filter objects and folders | Show Live objects only

Object sorting and filtering can make the Storage browser slower. For faster performance, select **Filter by name prefix only** from the filtering menu. Dismiss

Name	Size	Type	Created	Storage class	Last modified	Public access	Versic
data-ingestion_-11409897418611...	–	Folder	–	–	–	–	–
evaluation-component_58839251...	–	Folder	–	–	–	–	–
preprocessing-component_-86349...	–	Folder	–	–	–	–	–
splitting-component_5200078533...	–	Folder	–	–	–	–	–
training-component_-4023293503...	–	Folder	–	–	–	–	–

9. API - FastAPI

Flask

- Lightweight and simple web framework
- Easy to learn and flexible
- Requires manual setup for documentation
- Poor scalability for a high number of concurrent request
- Best suited for small projects and quick prototypes

FastAPI

- Modern, high-performance API framework
- Automatic API documentation (Swagger/OpenAPI)
- Can test the endpoints directly without curl by using Swagger UI
- Well-suited for scalable APIs and ML applications

Conclusion

⇒ FastAPI is preferred for this project due to its performance, built-in features, and suitability for deploying machine learning APIs.

9. API - HTTP endpoints

GET /health ● Check API and model status ● Used for diagnostics and monitoring	**POST /predict** ● Send input features (JSON) ● Returns model prediction
GET /past_predictions ● Retrieve history of predictions	**PUT /past_predictions/{id}** ● Update a past prediction by ID
GET /docs ● Interactive API documentation (Swagger UI) ● Test endpoints directly in browser	

9. API - deployment

🏠 ValoRe (house price predictor) API

📖 API Reference

Important: Please replace `SERVICE_URL` in the examples below with the official URL provided by your deployment (e.g., Cloud Run or Localhost).

`GET /health`

Diagnostic check for server and model status.

EXAMPLE:

```
curl -X GET SERVICE_URL/health
```

`POST /predict`

Submits 20 features to the ML pipeline. Expects a JSON object nested in a "`data`" key.

LINUX / APPLE / POWERSHELL

```
curl -X POST "SERVICE_URL/predict" -H "Content-Type: application/json" -d '{"data": {"id": "7229300521", "date": "20
141013T000000", "bedrooms": 2, "bathrooms": 1, "sqft_living": 1180, "sqft_lot": 5650, "floors": "1", "waterfront":
0, "view": 0, "condition": 3, "grade": 7, "sqft_above": 1180, "sqft_basement": 0, "yr_built": 1955, "yr_renovated":
0, "zipcode": "98178", "lat": 47.5112, "long": -122.257, "sqft_living15": 1340, "sqft_lot15": 5650}}'
```

WINDOWS (CMD)

```
curl -X POST "SERVICE_URL/predict" -H "Content-Type: application/json" -d "{\"data\": {\"id\": \"7229300521\", \"dat
e\": \"20141013T000000\", \"bedrooms\": 2, \"bathrooms\": 1, \"sqft_living\": 1180, \"sqft_lot\": 5650, \"floors\":
\"1\", \"waterfront\": 0, \"view\": 0, \"condition\": 3, \"grade\": 7, \"sqft_above\": 1180, \"sqft_basement\": 0,
\"yr_built\": 1955, \"yr_renovated\": 0, \"zipcode\": \"98178\", \"lat\": 47.5112, \"long\": -122.257, \"sqft_living
15\": 1340, \"sqft_lot15\": 5650}}"
```

`GET /past_predictions`

Retrieves the history of all predictions stored in memory.

EXAMPLE:

```
curl -X GET SERVICE_URL/past_predictions
```

`PUT /past_predictions/{id}`

Updates an existing record's price prediction by its ID.

LINUX / APPLE / POWERSHELL

```
curl -X PUT "SERVICE_URL/past_predictions/7229300521" -H "Content-Type: application/json" -d '{"prediction": 250000.
0}'
```

WINDOWS (CMD)

```
curl -X PUT "SERVICE_URL/past_predictions/7229300521" -H "Content-Type: application/json" -d "{\"prediction\": 25000
0.0}"
```

`GET /docs`

Interactive documentation for testing all endpoints.

Open Swagger UI →

9. API - deployment

King County house price predictor 1.1.0 OAS 3.1

/openapi.json

An API to predict housing prices using a model fetched from GCS.

default

- GET / Welcome Page
- POST /predict Predict
- GET /past_predictions Get Past Predictions
- PUT /past_predictions/{record_id} Update Prediction
- GET /health Health

Schemas

- HTTPValidationError > Expand all object
- HouseData > Expand all object
- PredictionPayload > Expand all object
- UpdatePayload > Expand all object
- ValidationError > Expand all object

9. API - Local uvicorn deployment

GET /health

POST /predict

GET /past_predictions

POST /predict

GET /past_predictions

PUT /past_predictions/id

GET /past_predictions

```
C:\Windows\System32\cmd.e
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>curl -X GET http://localhost:8
000/health
{"status":"healthy","model_loaded":true,"model_path":"gs://mlsd-valore-2026-
0001/pipeline_root_valore/826612837630/valore-pipeline-20260331180311/traini
ng-component_-4023293503378292736/model"}
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>curl -X POST "http://localhost
:8000/predict" -H "Content-Type: application/json" -d "{\"data\": {\"id\": \
"7229300521\", \"date\": \"20141013T000000\", \"bedrooms\": 2, \"bathrooms\"
: 1, \"sqft_living\": 1180, \"sqft_lot\": 5650, \"floors\": \"1\", \"waterfr
ont\": 0, \"view\": 0, \"condition\": 3, \"grade\": 7, \"sqft_above\": 1180,
 \"sqft_basement\": 0, \"yr_built\": 1955, \"yr_renovated\": 0, \"zipcode\":
 \"98178\", \"lat\": 47.5112, \"long\": -122.257, \"sqft_living15\": 1340, \
"sqft_lot15\": 5650}}"
{"prediction":234605.59375,"currency":"USD","status":"success"}
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>curl -X GET http://localhost:8
000/past_predictions
[{"id":"7229300521","prediction":234605.59375,"currency":"USD"}]
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>curl -X POST "http://localhost
:8000/predict" -H "Content-Type: application/json" -d "{\"data\": {\"id\": \
"9876543210\", \"date\": \"20150225T000000\", \"bedrooms\": 5, \"bathrooms\"
: 4.5, \"sqft_living\": 4500, \"sqft_lot\": 12000, \"floors\": \"2\", \"wate
rfront\": 1, \"view\": 4, \"condition\": 5, \"grade\": 12, \"sqft_above\": 4
000, \"sqft_basement\": 500, \"yr_built\": 2010, \"yr_renovated\": 0, \"zipc
ode\": \"98039\", \"lat\": 47.6298, \"long\": -122.2301, \"sqft_living15\":
4200, \"sqft_lot15\": 11500}}"
{"prediction":3691717.5,"currency":"USD","status":"success"}
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>curl -X GET http://localhost:8
000/past_predictions
[{"id":"7229300521","prediction":234605.59375,"currency":"USD"},{"id":"98765
43210","prediction":3691717.5,"currency":"USD"}]
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>curl -X PUT "http://localhost:
8000/past_predictions/7229300521" -H "Content-Type: application/json" -d "{\
"prediction\": 250000.0}"
{"message":"Updated successfully","data":{"id":"7229300521","prediction":250
000.0,"currency":"USD"}}
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>curl -X GET http://localhost:8
000/past_predictions
[{"id":"7229300521","prediction":250000.0,"currency":"USD"},{"id":"987654321
0","prediction":3691717.5,"currency":"USD"}]
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>
```

```
C:\Windows\System32\cmd.e
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>set MODEL_PATH=gs://mlsd-valor
e-2026-0001/pipeline_root_valore/826612837630/valore-pipeline-20260331180311
/training-component_-4023293503378292736/model

C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>uv run uvicorn src.api.app:app
 --reload --port 8000
INFO:     Will watch for changes in these directories: ['C:\\ANT\\Data scien
ce\\M1\\Q2\\MLops\\Projet\\ValoRe']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [73644] using StatReload
INFO:     Started server process [6844]
INFO:     Waiting for application startup.
Attempting to load model from: gs://mlsd-valore-2026-0001/pipeline_root_valo
re/826612837630/valore-pipeline-20260331180311/training-component_-402329350
3378292736/model
Model loaded successfully from GCS!
INFO:     Application startup complete.
INFO:     127.0.0.1:60268 - "GET /health HTTP/1.1" 200 OK
INFO:     127.0.0.1:60270 - "POST /predict HTTP/1.1" 200 OK
INFO:     127.0.0.1:60273 - "GET /past_predictions HTTP/1.1" 200 OK
INFO:     127.0.0.1:61981 - "POST /predict HTTP/1.1" 200 OK
INFO:     127.0.0.1:61983 - "GET /past_predictions HTTP/1.1" 200 OK
INFO:     127.0.0.1:60535 - "PUT /past_predictions/7229300521 HTTP/1.1" 200
OK
INFO:     127.0.0.1:60537 - "GET /past_predictions HTTP/1.1" 200 OK
```

9. API - Local docker deployment

GET /health

POST /predict

GET /past_predictions

POST /predict

GET /past_predictions

PUT /past_predictions/id

GET /past_predictions

```
C:\Windows\System32\cmd.e

C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>curl -X GET http://localhost:7
000/health
{"status":"healthy","model_loaded":true,"model_path":"gs://mlsd-valore-2026-
0001/pipeline_root_valore/826612837630/valore-pipeline-20260331180311/traini
ng-component_-4023293503378292736/model"}
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>curl -X POST "http://localhost
:7000/predict" -H "Content-Type: application/json" -d "{\"data\": {\"id\": \
"7229300521\", \"date\": \"20141013T000000\", \"bedrooms\": 2, \"bathrooms\"
: 1, \"sqft_living\": 1180, \"sqft_lot\": 5650, \"floors\": \"1\", \"waterfr
ont\": 0, \"view\": 0, \"condition\": 3, \"grade\": 7, \"sqft_above\": 1180,
 \"sqft_basement\": 0, \"yr_built\": 1955, \"yr_renovated\": 0, \"zipcode\":
 \"98178\", \"lat\": 47.5112, \"long\": -122.257, \"sqft_living15\": 1340, \
"sqft_lot15\": 5650}}"
{"prediction":234605.59375,"currency":"USD","status":"success"}
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>curl -X GET http://localhost:7
000/past_predictions
[{"id":"7229300521","prediction":234605.59375,"currency":"USD"}]
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>curl -X POST "http://localhost
:7000/predict" -H "Content-Type: application/json" -d "{\"data\": {\"id\": \
"9876543210\", \"date\": \"20150520T000000\", \"bedrooms\": 4, \"bathrooms\"
: 3.5, \"sqft_living\": 3800, \"sqft_lot\": 12000, \"floors\": 2.0, \"waterf
ront\": 1, \"view\": 4, \"condition\": 4, \"grade\": 11, \"sqft_above\": 330
0, \"sqft_basement\": 500, \"yr_built\": 2008, \"yr_renovated\": 0, \"zipcod
e\": \"98004\", \"lat\": 47.6103, \"long\": -122.2007, \"sqft_living15\": 35
00, \"sqft_lot15\": 11000}}"
{"prediction":2838269.75,"currency":"USD","status":"success"}
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>curl -X GET http://localhost:7
000/past_predictions
[{"id":"7229300521","prediction":234605.59375,"currency":"USD"},{"id":"98765
43210","prediction":2838269.75,"currency":"USD"}]
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>curl -X PUT "http://localhost:
7000/past_predictions/7229300521" -H "Content-Type: application/json" -d "{\
"prediction\": 250000.0}"
{"message":"Updated successfully","data":{"id":"7229300521","prediction":250
000.0,"currency":"USD"}}
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>curl -X GET http://localhost:7
000/past_predictions
[{"id":"7229300521","prediction":250000.0,"currency":"USD"},{"id":"987654321
0","prediction":2838269.75,"currency":"USD"}]
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>
```

```
C:\Windows\System32\cmd.e

C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>set MODEL_PATH=gs://mlsd-valor
e-2026-0001/pipeline_root_valore/826612837630/valore-pipeline-20260331180311
/training-component_-4023293503378292736/model

C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>docker run -p 7000:8080 -e MOD
EL_PATH=%MODEL_PATH% -e GOOGLE_CLOUD_PROJECT=valore-mlsd-project -v "%AppDat
a%\gcloud:/root/.config/gcloud" valore
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
Attempting to load model from: gs://mlsd-valore-2026-0001/pipeline_root_valo
re/826612837630/valore-pipeline-20260331180311/training-component_-402329350
3378292736/model
Model loaded successfully from GCS!
INFO:     172.17.0.1:45178 - "GET /health HTTP/1.1" 200 OK
INFO:     172.17.0.1:42746 - "GET /health HTTP/1.1" 200 OK
INFO:     172.17.0.1:43192 - "POST /predict HTTP/1.1" 200 OK
INFO:     172.17.0.1:34678 - "GET /past_predictions HTTP/1.1" 200 OK
INFO:     172.17.0.1:34124 - "POST /predict HTTP/1.1" 200 OK
INFO:     172.17.0.1:34138 - "GET /past_predictions HTTP/1.1" 200 OK
INFO:     172.17.0.1:50466 - "PUT /past_predictions/7229300521 HTTP/1.1" 200
 OK
INFO:     172.17.0.1:50474 - "GET /past_predictions HTTP/1.1" 200 OK
```

9. API - Google cloud deployment

Service url : https://valore-826612837630.europe-west1.run.app/

Google Cloud | VaLoRe-MLSD-project | cloud | Recherche

Cloud Run

- Présentation
- Services
- Jobs
- Pools de nœuds de calcul
- Mappages de domaines
- Notes de version

Informations sur le service | Modifier et déployer la nouvelle révision | Se connecter au dépôt | Test | Apprendre | Actualiser

valore | Région : europe-west1 | URL :https://valore-826612837630.europe-west1.run.app | Scaling : automatique (minimum : 0, maximum : 5)

Observabilité | Révisions | Source | Déclencheurs | Réseau | Sécurité | YAML

Révisions | Gérer le trafic

Filtrer | Filtrer les révisions

Nom	Trafic	Dépl	Actions
valore-00002-p9z	100 % (vers la révision la plus récente)	il y a	

Nombre maximal d'instances pour la révision	5
Métriques d'autoscaling	Cibles d'autoscaling par défaut
Image	europe-west1-docker.pkg.dev/valore-mlsd-project/...
Port	8080
Build	(aucune information disponible concernant le build)
Source	gs://run-sources-valore-mlsd-project-europe-west...
Commande et arguments	(point d'entrée du conteneur)
Limite de processeur	1
Limite de mémoire	512MiB

Variables d'environnement (1)

Nom	Valeur
MODEL_PATH	gs://mlsd-valore-2026-0001/pipeline_root_valore/826612837630/valore pipeline-20260331180311/training-component_-4023293503378292736/model

Volumes (0)

Vérifications de l'état (1)

Connexions Cloud SQL (0)

9. API - Google cloud deployment

GET /health

POST /predict

GET /past_predictions

POST /predict

GET /past_predictions

PUT /past_predictions/id

GET /past_predictions

C:\Windows\System32\cmd.e

```
C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>echo. & curl -X GET https://valore-826612837630.europe-west1.run.app/health &
 echo.

{"status":"healthy","model_loaded":true,"model_path":"gs://mlsd-valore-2026-0001/pipeline_root_valore/826612837630/valore-p
ipeline-20260331180311/training-component_-4023293503378292736/model"}

C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>echo. & curl -X POST "https://valore-826612837630.europe-west1.run.app/predic
t" -H "Content-Type: application/json" -d "{\"data\": {\"id\": \"7229300521\", \"date\": \"20141013T000000\", \"bedrooms\":
 2, \"bathrooms\": 1, \"sqft_living\": 1180, \"sqft_lot\": 5650, \"floors\": \"1\", \"waterfront\": 0, \"view\": 0, \"condi
tion\": 3, \"grade\": 7, \"sqft_above\": 1180, \"sqft_basement\": 0, \"yr_built\": 1955, \"yr_renovated\": 0, \"zipcode\":
\"98178\", \"lat\": 47.5112, \"long\": -122.257, \"sqft_living15\": 1340, \"sqft_lot15\": 5650}}" & echo.

{"prediction":234605.59375,"currency":"USD","status":"success"}

C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>echo. & curl -X GET https://valore-826612837630.europe-west1.run.app/past_pre
dictions & echo.

[{"id":"7229300521","prediction":234605.59375,"currency":"USD"},{"id":"8888888888","prediction":950000.0,"currency":"USD"}]

C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>echo. & curl -X POST "https://valore-826612837630.europe-west1.run.app/predic
t" -H "Content-Type: application/json" -d "{\"data\": {\"id\": \"8888888888\", \"date\": \"20151111T000000\", \"bedrooms\":
 4, \"bathrooms\": 3, \"sqft_living\": 3200, \"sqft_lot\": 15000, \"floors\": \"2\", \"waterfront\": 1, \"view\": 2, \"cond
ition\": 5, \"grade\": 10, \"sqft_above\": 2800, \"sqft_basement\": 400, \"yr_built\": 1998, \"yr_renovated\": 2015, \"zipc
ode\": \"98001\", \"lat\": 47.2974, \"long\": -122.228, \"sqft_living15\": 3000, \"sqft_lot15\": 14000}}" & echo.

{"prediction":961429.8125,"currency":"USD","status":"success"}

C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>echo. & curl -X GET https://valore-826612837630.europe-west1.run.app/past_pre
dictions & echo.

[{"id":"7229300521","prediction":234605.59375,"currency":"USD"},{"id":"8888888888","prediction":961429.8125,"currency":"USD
"}]

C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>echo. & curl -X PUT "https://valore-826612837630.europe-west1.run.app/past_pr
edictions/8888888888" -H "Content-Type: application/json" -d "{\"prediction\": 950000.0}" & echo.

{"message":"Updated successfully","data":{"id":"8888888888","prediction":950000.0,"currency":"USD"}}

C:\ANT\Data science\M1\Q2\MLops\Projet\ValoRe>echo. & curl -X GET https://valore-826612837630.europe-west1.run.app/past_pre
dictions & echo.

[{"id":"7229300521","prediction":234605.59375,"currency":"USD"},{"id":"8888888888","prediction":950000.0,"currency":"USD"}]
```

9. API - Pytest

- Pytest : ‘tests/test_api_app.py’
- Goal
  - Validate that API endpoints work correctly
  - Ensure model predictions and error handling behave as expected

9. API - Pytest

1. Prediction Success ● Sends valid toy data to `/predict` ● Checks: ○ Status = 200 ○ Correct prediction value	**2. Model Error Handling** ● Simulates missing model ● Checks: ○ Status = 500 ○ Proper error message returned
3. Health Check ● Calls `/health` ● Verifies: ○ Model is loaded	**4. Retrieve History** ● Calls `/past_predictions` ● Checks: ○ Stored predictions are returned correctly

9. API - Improvements

- Current situation : model modification ⇒ API needs to be redeployed
- Future : automate API deployment when there is a model pipeline modification.

Thank you for your listening!